

MyEscrow developer meeting notes

What I want from this meeting

Give the developer a clear picture of what MyEscrow is, what has already been built, and where expert technical judgment is now needed.

The desired outcome is agreement on:

- the true current state of the product;
- the biggest technical and product risks;
- what should be stabilized before more features are added;
- a practical development plan with clear phases.

Opening summary

MyEscrow is a milestone-based transaction platform for buyers and sellers.

A buyer and seller agree on a transaction, divide it into milestones, sign the agreement, fund it, and release funds as work is approved. The product also includes change requests, disputes, cancellation flows, notifications, and an internal operations area.

I have built the current version iteratively and largely through AI-assisted development. It is a functional staging MVP, but I now want an experienced developer to assess it, establish a sound technical foundation, and help determine what is required for a production product.

What currently works

- Individual and business account signup
- Email verification, login, password reset, and password change
- Creating an escrow as either the buyer or seller
- Inviting a counterparty by email
- Agreement review, signatures, rejection, and requested changes
- Milestone creation and ordered milestone completion
- Funding from an internal demonstration wallet
- Seller work submission and buyer approval or revision requests
- Milestone-specific disputes and proposed settlements
- Mutual and unilateral cancellation requests
- Notifications, transaction history, and financial ledger history
- An operations area for alerts, failed jobs, account roles, and recovery actions
- Automated background processing for reminders, expiries, retries, and reconciliation

The technical picture—in brief

The product is split into:

- a Next.js/React web application;
- a Node.js/Fastify API;
- PostgreSQL with Prisma;
- a background operations worker;
- Resend for email;
- Vercel for the frontend;
- an Oracle-hosted API and database environment.

The frontend and backend are separate Git repositories, with the frontend included in the main repository as a submodule.

The application has automated tests and build checks. At the time of the technical review, 36 API tests and 38 frontend tests were passing.

Detailed architecture, data-model, API, deployment, and limitation notes are in [MYESCROW_TECHNICAL_OVERVIEW.md](#).

Important context to be clear about

It does not currently move real money

The wallet, top-ups, withdrawals, funding, releases, and refunds are database-based simulations. There is no connection to a bank, payment processor, regulated custodian, or real settlement system.

The internal ledger is designed to account for movements safely, but it is not evidence that real funds are held.

It is not production-ready

The staging product demonstrates the intended workflows, but production use would require work in areas including:

- real payment and custody integration;
- security hardening;
- identity and regulatory compliance;
- evidence/file storage;
- monitoring and controlled deployment;
- final dispute and arbitration policies;
- broader end-to-end testing.

Some parts evolved faster than the overall architecture

The application includes meaningful safety mechanisms—agreement versions, signatures, idempotent financial commands, ledger reconciliation, audit events, and retryable background jobs.

At the same time, some core files have become very large, frontend and backend types are not fully shared, status handling is spread across the codebase, and two different dashboard experiences exist with different levels of functionality.

What I would like the developer to assess

1. Is the overall architecture reasonable for the intended product?
2. Which parts are solid enough to keep?
3. Which parts should be refactored or replaced before further development?
4. Are the ledger, agreement, milestone, dispute, and cancellation models conceptually sound?
5. What are the most serious security or data-integrity risks?

6. What would be required to integrate a real payment or escrow provider?
7. Should the two frontend experiences be consolidated?
8. Is the current deployment model appropriate, or should it be simplified?
9. What testing and monitoring are missing?
10. What is a realistic path from the current MVP to a controlled pilot?

Questions I want answered

- What concerns you most after reviewing the code?
- Where has the current implementation become unnecessarily complex?
- Are there any parts you would avoid building on?
- What should be addressed before adding another major feature?
- What information or product decisions do you need from me?
- How would you divide the work into immediate, near-term, and later phases?
- Which work requires specialist help outside normal application development?
- What level of effort would you expect for stabilization versus production readiness?

Product decisions that may affect the technical design

These do not all need to be answered in the first meeting, but the developer should know they are still open:

- Which countries and currencies will be supported?
- Who will legally hold customer funds?
- Which payment or escrow provider will be used?
- What identity checks are required for individuals and businesses?
- What happens when a buyer does not respond to completed work?
- Who decides a dispute if the parties cannot agree?
- Can dispute decisions be appealed?
- When can either party cancel?
- Are fees charged, and when?
- What evidence can users submit, and how long is it retained?

Suggested meeting flow

1. Product explanation — 5 minutes

Explain the buyer/seller problem, the milestone model, and the intended user outcome.

2. Demonstration — 10–15 minutes

Show:

1. account creation or login;
2. escrow creation;
3. agreement and milestones;
4. counterparty approval;
5. funding;
6. milestone submission and approval;
7. dispute or cancellation;
8. operations alerts.

Keep the demonstration focused on the workflow rather than every screen.

3. Current technical shape — 5 minutes

Explain the frontend, API, database, worker, email provider, and staging hosting at a high level.

Be explicit that money is simulated.

4. Developer assessment — 15–20 minutes

Ask the developer to describe:

- immediate concerns;
- what should be preserved;
- what needs deeper review;
- how they would approach stabilization.

5. Next steps — 5 minutes

Agree on:

- what access or documentation the developer needs;
- what they will review;
- what assessment or plan they will return with;
- the next decision meeting.

What not to get lost in during the meeting

- Individual database tables
- Every API endpoint
- Minor interface styling
- Historical implementation details
- Detailed infrastructure credentials
- Solving every regulatory or product-policy question immediately

The detailed technical overview is available for follow-up. The meeting should establish shared understanding and identify the next set of decisions.

Closing statement

The current version proves that the core experience can be represented as a working product. I am not asking the developer simply to continue adding features. I want them to evaluate the foundation, identify what can be trusted, and help turn the MVP into a deliberately engineered product.